

Large Language Models

Jorge Gallego

Inter-American Development Bank

Bogotá Summer School in Economics — 2026

Agenda de la sesión

- Cerramos el curso con el tema que hoy domina: los **Large Language Models** (LLMs)

1. ¿Qué son y por qué ahora?
2. De texto a números: tokens y embeddings
3. De las RNN a los Transformers
4. ¿Cómo se entrena un LLM?
5. De modelo base a asistente
6. Cómo se usa (sin reentrenar)
7. Límites, riesgos y política pública

Construye sobre lo visto en S5 (embeddings) y S6 (redes neuronales, RNN, atención).

Parte 1

¿Qué son y por qué ahora?

¿Qué es un LLM?

- Un **modelo de lenguaje** es, en esencia, un sistema que **predice texto**: dado lo anterior, ¿qué palabra sigue?
- Un LLM es uno **muy grande**, entrenado con cantidades enormes de texto (libros, web, código. . .)
- Ejemplos: ChatGPT, Claude, Gemini, Llama
- La sorpresa: predecir bien la siguiente palabra, a esta escala, produce algo que *parece* entender y razonar

¿Por qué explotaron justo ahora?

- No es magia nueva: es la combinación de tres ingredientes que maduraron juntos
1. **Arquitectura:** el *Transformer* (2017), que escala muy bien
 2. **Datos:** prácticamente todo el texto digitalizado
 3. **Cómputo:** GPUs masivas para entrenar modelos gigantes
- Resultado: un salto enorme en pocos años (de autocompletar torpe a asistentes conversacionales)

La idea central

- Todo se reduce a una tarea sorprendentemente simple:

predecir la siguiente palabra, una y otra vez

- El modelo genera texto de forma **autoregresiva**: predice una palabra, la agrega, y vuelve a predecir
- Toda la conversación, el resumen o el código surgen de repetir ese paso

Conexión con el curso

- Un LLM **no** es un tema aparte: es *machine learning* llevado al extremo
- Es una **red neuronal profunda** (S6), entrenada para una tarea de **predicción** (clasificar la siguiente palabra entre miles)
- Usa **representaciones** aprendidas de las palabras (embeddings, S5)
- Se ajusta minimizando una **función de pérdida** (S3, S4)
Mismos principios, escala distinta.

Parte 2

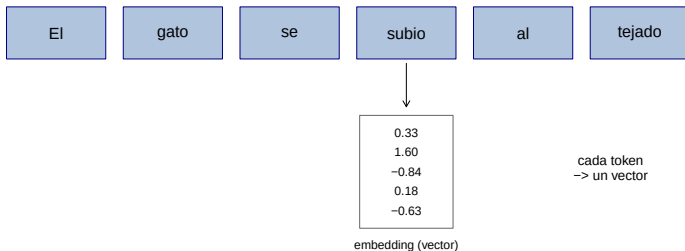
De texto a números: tokens y embeddings

Las máquinas no leen palabras

- Un modelo solo opera con **números**. Primero hay que convertir el texto
- Paso 1 — **tokenización**: partir el texto en piezas (*tokens*), normalmente sub-palabras
- Paso 2 — **embedding**: asignar a cada token un **vector** de números
- Esos vectores son la “entrada” de la red

Tokens y embeddings

De texto a numeros: tokens y embeddings



Los tokens no siempre son palabras completas; pueden ser fragmentos ("sub", "ió").

Embeddings: significados como vectores

- Ya los vimos en S5: palabras con significados parecidos quedan **cerca** en el espacio vectorial
- Capturan relaciones: el famoso $rey - hombre + mujer \approx reina$
- En un LLM estos vectores **no** se fijan a mano: se **aprenden** durante el entrenamiento
- Son el “vocabulario interno” con el que el modelo piensa

Parte 3

De las RNN a los Transformers

Recordemos las RNN (S6)

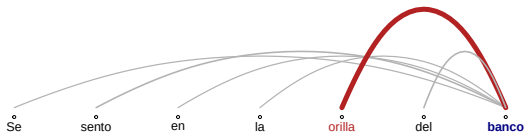
- Las redes recurrentes procesan el texto **palabra por palabra**, arrastrando una “memoria”
- Problema: con secuencias largas **olvidan** lo lejano (el gradiente que se desvanece)
- Y son **lentas**: al ir en orden, no se pueden paralelizar bien
- Necesitábamos otra forma de mirar la secuencia

La idea de atención

- En vez de leer en orden y olvidar, **mirar todas las palabras a la vez** y decidir a cuáles “prestar atención”
- Para entender una palabra, el modelo **pondera** las demás según cuán relevantes son

Atencion: la palabra 'banco' mira a las demas

Lineas mas gruesas = mas atencion



Self-attention: la intuición

- Cada palabra se “actualiza” mirando a las otras de la frase
- Ejemplo: en “*el banco de la plaza*” vs. “*el banco me dio el crédito*”, la palabra **banco** significa cosas distintas
- La atención deja que el **contexto** (plaza vs. crédito) defina el significado
- Así el modelo resuelve ambigüedad y relaciones a larga distancia

La atención, en una ecuación

Cada palabra emite una **consulta** (Q), y cada palabra ofrece una **clave** (K) y un **valor** (V):

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}}\right) V$$

- $QK^{\top}$: qué tan compatible es cada palabra con cada otra (similitud)
- *softmax*: convierte esas compatibilidades en **pesos** que suman 1
- El resultado es un promedio **ponderado** de los valores: cada palabra “recoge” información de las relevantes

El Transformer

- Un **Transformer** apila muchas capas de atención (más redes neuronales entre ellas)
- Procesa toda la secuencia **en paralelo** — por eso escala tan bien con datos y cómputo
- Es la arquitectura detrás de **todos** los LLMs actuales

El artículo que lo introdujo (2017) se titula, con razón, *“Attention is all you need”*.

Parte 4

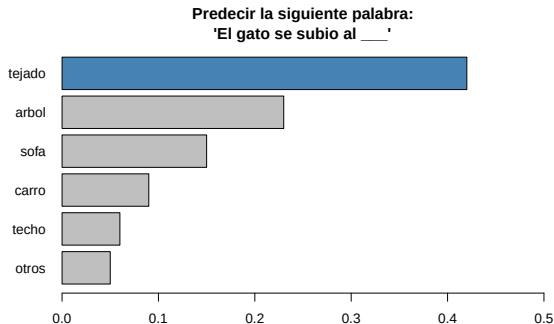
¿Cómo se entrena un LLM?

Pre-entrenamiento auto-supervisado

- Tomamos texto y le tapamos la siguiente palabra; el modelo intenta adivinarla
- No necesita etiquetas humanas: **el texto es su propia respuesta** (por eso “auto-supervisado”)
- Repetido billones de veces, el modelo aprende gramática, hechos y estilo

Recordemos S1: clasificar. Aquí la “clase” es *cuál de todas las palabras del vocabulario sigue*.

Predecir = asignar probabilidades



El modelo produce una probabilidad para **cada** palabra posible:

$$P(w_t \mid w_1, \dots, w_{t-1}) = \frac{e^{z_{w_t}}}{\sum_v e^{z_v}} \quad (\textit{softmax})$$

La función de pérdida

- Entrenar = ajustar los parámetros para que la palabra **correcta** reciba alta probabilidad
- Se minimiza la **entropía cruzada** (penaliza la “sorpresa”):

$$\mathcal{L} = - \sum_t \log P(w_t \mid w_{<t})$$

- Es la misma lógica de la **máxima verosimilitud** del logit (S3), a gran escala
- Se optimiza con **descenso de gradiente** (S6)

Parte 5

De modelo base a asistente

Un modelo “crudo” no es un chatbot

- Tras el pre-entrenamiento, el modelo solo sabe **continuar texto**
- Si le escribes una pregunta, podría responder... o seguir con más preguntas (porque eso también es “texto plausible”)
- Para que sea un **asistente útil** hay que **alinearlo** con lo que queremos
- Eso se hace en dos fases adicionales

Parte 6

Cómo se usa (sin reentrenar)

Prompting e *in-context learning*

- No hace falta reentrenar para cada tarea: basta con **pedírselo bien** (el *prompt*)
- El modelo puede aprender del **ejemplo dentro del propio prompt** (*in-context learning*)
- **Few-shot**: darle unos ejemplos resueltos antes de la tarea real
- **Chain-of-thought**: pedirle que “piense paso a paso” mejora el razonamiento

Conectarlo a tus datos: embeddings y RAG

- Un LLM no conoce *tus* documentos ni lo más reciente
- **RAG** (*retrieval-augmented generation*): primero **recuperar** los textos relevantes, luego dejar que el modelo responda con ellos
- La recuperación usa **embeddings** (S5): buscar por **significado**, no por palabra exacta
- Reduce alucinaciones y permite citar fuentes

Agentes y herramientas

- Un LLM puede decidir **usar herramientas**: buscar en la web, ejecutar código, consultar una base de datos
- Encadenando pasos, se vuelve un **agente** que resuelve tareas de varios pasos
- Así supera dos límites: el conocimiento desactualizado y el cálculo exacto

Parte 7

Límites, riesgos y política pública

Límites y riesgos

- **Alucinaciones:** inventan información con total seguridad
- **Sesgos:** reflejan los de sus datos de entrenamiento
- **Desactualización:** solo “saben” hasta su fecha de corte
- **Costo y ambiente:** entrenar y operar es caro y consume energía
- **Privacidad y propiedad:** datos sensibles y derechos de autor

¿Por qué alucinan?

- Un LLM optimiza para generar texto **plausible**, no para consultar una base de hechos
- Si no “sabe”, igual produce algo que *suen*a bien
- Por eso: **verificación humana** y, cuando importa, **RAG** con fuentes

Regla de oro: úsalo como un asistente brillante pero falible, no como un oráculo.

LLMs en política pública y economía

- **Análisis de texto a escala:** leyes, sentencias, noticias, redes sociales
- **Codificar datos cualitativos:** clasificar miles de PQRS o respuestas abiertas de encuestas (recuerda S1)
- **Atención al ciudadano:** asistentes para trámites e información
- **Apoyo a la investigación:** resumir literatura, primeras versiones de código

Cautelas en lo público

- Las decisiones públicas exigen **transparencia** y **rendición de cuentas** — difícil con modelos opacos
- **Verificación humana** obligatoria en decisiones que afectan derechos
- Vigilar **sesgos y equidad** (como discutimos en S4)
- Cuidado con **datos personales** y confidenciales

Resumen de la sesión

- Un LLM **predice la siguiente palabra**, a una escala que produce capacidades asombrosas
- Detrás hay **Transformers** y **atención**; antes, **tokens** y **embeddings**
- Se **pre-entrena** prediciendo texto y se **alinea** para ser útil
- Es poderoso pero **falible**: alucina, sesga, hay que verificarlo
- Y sobre todo: es el **mismo** *machine learning* del curso — representación, predicción, optimización y evaluación — a gran escala